NLU course project (Language Modeling part)

Gabriele Masciulli (258394)

University of Trento

gabriele.masciulli@studenti.unitn.it

1. AI Usage Declaration

This project utilized AI assistance for boilerplate code generation, general project structure setup, and helped in writing the NT-ASGD optimizer implementation. All core algorithmic development, experimental design, and analysis were conducted independently.

2. Introduction

This report outlines the progressive refinement of a language model based on LSTM architectures. The work was conducted in two stages: (1.A) a sequence of baseline enhancements and (1.B) integration of advanced regularization techniques. The goal was to minimize test perplexity (PPL), with emphasis on the regularization phase, where improvements were guided by techniques from the AWD-LSTM framework.

3. Implementation Details

3.1. Part 1.A: Baseline Enhancements

The RNN-based architecture was first substituted with a multi-layer LSTM and conducted extensive hyperparameter tuning across embedding sizes, hidden dimensions, and layer counts. Manual learning rate scheduling improved convergence stability.

Next, two dropout layers were introduced following Zaremba et al. [1] (2014): one after the embedding layer and one before the final projection layer. This reduced overfitting. Finally, SGD was replaced with AdamW, which accelerated convergence but led to a slightly worse final test perplexity as can be seen from A. These experiments highlighted the trade-off between training speed and final model performance.

3.2. Part 1.B: Advanced Regularization

Building on the best LSTM baseline, three regularization techniques were integrated based on Merity et al. [2] (2017). First, **Weight Tying** was implemented by ensuring the output dimension of the final LSTM layer matched the embedding dimension, allowing direct tying. This approach was chosen after initially considering tying to an intermediate LSTM hidden size, which would have required an additional projection layer.

Second, **Variational Dropout** was integrated using the LockedDropout technique, inspired by Merity et al. [3] (2017). An instance of LockedDropout was applied to the word embeddings after the embedding layer. Furthermore, distinct LockedDropout instances were applied to the outputs of each LSTM layer before these outputs were passed to the next LSTM layer or the final linear layer. Each mask was sampled once per sequence and kept the same across timesteps to preserve temporal consistency, unlike standard dropout. The dropout mask was scaled by $\frac{1}{(1-p)}$, preventing both a mismatch in the activations between training and inference time and vanishing gradient in deeper layers.

The final requirement i.e. optimization strategy involved a custom implementation of **Non-monotonically Triggered Averaged SGD** (NT-ASGD). Training began with a standard SGD optimizer. A switch to the custom NT-AvSGD optimizer is triggered based on a non-monotonic condition. While the original paper suggests switching after a simple patience-based trigger (e.g.: 5 steps of no improvement), the one adopted in this work, following the actual implementation of the paper by Merity et al. involves triggering the switch only if the current validation perplexity becomes worse than the best perplexity recorded in a historical window, excluding the most recent 'nonmono' epochs. This approach makes sure the model has more definitely moved past a simple performance plateau.

Upon switching, the NT-AvSGD optimizer is initialized with the model's current weights from the SGD phase and during evaluation epochs, the model's standard weights are temporarily swapped with the optimizer's averaged weights to compute the validation perplexity. The original (non-averaged) weights are then restored to continue the training step.

The weight averaging is performed as follows:

$$\mathbf{a}_k = \mathbf{a}_{k-1} + \frac{\mathbf{w}_k - \mathbf{a}_{k-1}}{k - T + 1} \tag{1}$$

where:

k is the current iteration in the training process, $\mathbf{a}_k$ is the averaged weight vector at optimization step k, representing the running average of the model parameters used for evaluation after averaging is triggered, $\mathbf{a}_{k-1}$ is the averaged weight vector at the previous optimization step $k-1$, $\mathbf{w}_k$is the current model weight vector at optimization step k and T is the optimization step at which weight averaging begins, triggered by the non-monotonic condition based on validation perplexity.

Finally, an experiment was conducted using pre-trained **GloVe embeddings** [4] to initialize the embedding layer, aiming to leverage external knowledge for potential performance gains.

Throughout all stages, extensive hyperparameter optimization was performed, focusing on learning rates, dropout probabilities, weight decay, and optimizer-specific parameters like the non-monotonic trigger point.

4. Results

Table 1 summarizes the test PPL achieved after implementing each requirement. Each technique significantly boosted performance beyond the improvements from baseline tuning.

Table 1: *Test Perplexity across model variants.*

Run	Description	Test PPL
A1	Base LSTM + tuning	140.90
A2	+ Zaremba-style dropout	104.28
A3	+ AdamW optimizer	111.39
B1	Weight Tying only	92.52
B2	+ Variational Dropout	89.92
B3	+ NT-ASGD (final)	77.34
(Opt)	+ GloVe embeddings	**75.59**

Each enhancement led to a consistent improvement in model performance. NT-ASGD provided the largest single improvement, confirming the utility of adaptive training dynamics and model averaging.

5. References

[1] W. Zaremba, I. Sutskever, and O. Vinyals, "Recurrent neural network regularization," 2014. [Online]. Available: https://arxiv.org/abs/1409.2329

[2] S. Merity, N. S. Keskar, and R. Socher, "Regularizing and optimizing lstm language models," *arXiv preprint arXiv:1708.02182*, 2017.

[3] S. Merity, "locked_dropout.py - awd-lstm-lm," https://github.com/salesforce/awd-lstm-lm/blob/master/locked_dropout.py, 2018, accessed: 2025-07-01.

[4] J. Pennington, R. Socher, and C. D. Manning, "Glove: Global vectors for word representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2014, pp. 1532–1543.

A. Part 1A: Validation Perplexity Curve

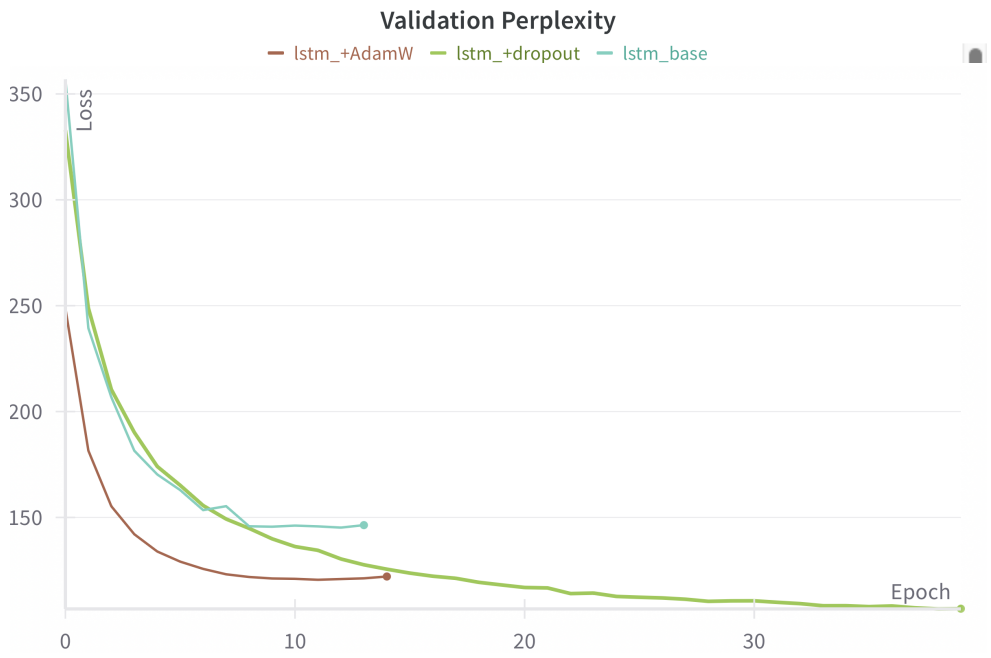


Figure 1: *Validation Perplexity vs Epochs. AdamW clearly shows faster convergence but achieves worse performance overall.*

B. Part 1B: Validation Perplexity Curve

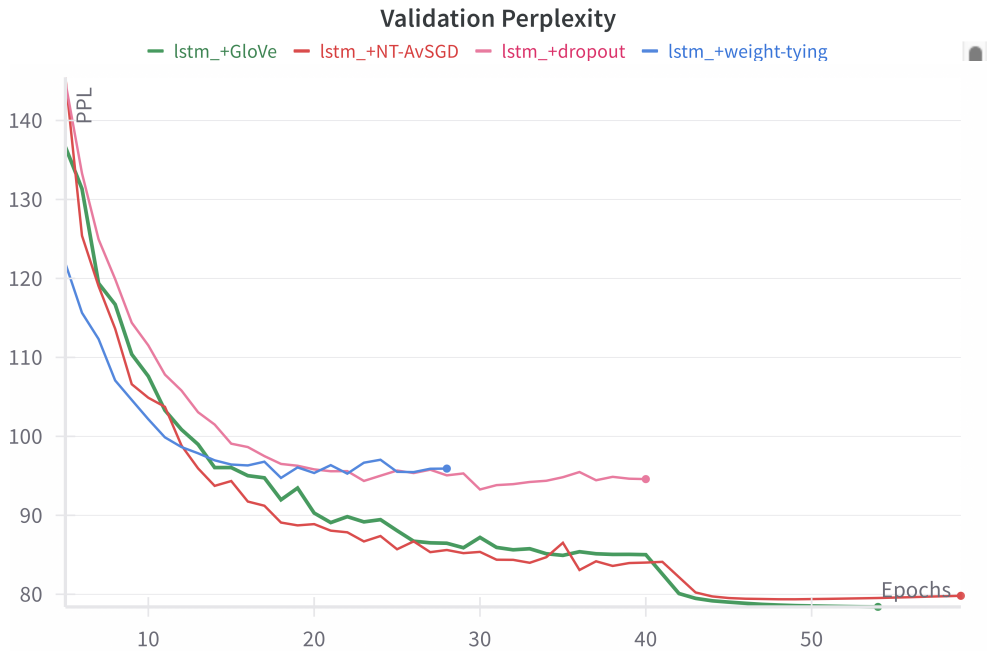
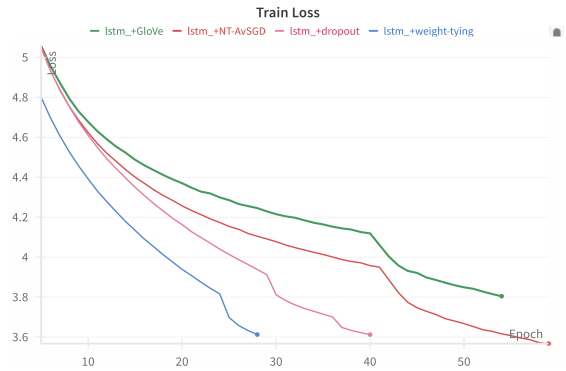
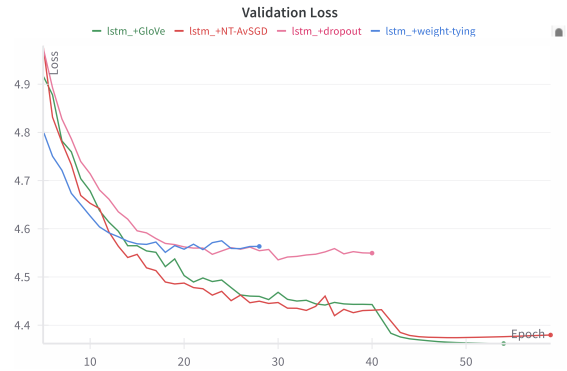


Figure 2: *Validation Perplexity vs Epochs. The sharp drop reflects the point at which NT-ASGD was triggered.*

C. Part 1B: Training and Validation Loss Trends



(a) *Training Loss vs Epochs for all models. Simpler models tend to achieve lower training loss but overfit quickly.*



(b) *Validation Loss vs Epochs for all models. More advanced models generalize better and maintain lower validation loss.*

D. Appendix: Hyperparameter Settings for Each Model Variant

Table 2: *Model Hyperparameters for Each Run*

Run	Description	Hyperparameters
A1	Base LSTM + tuning	emb_size=400, hid_size=650, n_layers=1, optimizer=SGD, lr=1.0, batch_size=64, epochs=30
A2	+ Zaremba-style dropout	emb_size=650, hid_size=650, n_layers=2, dropout (embed+pre-proj), optimizer=SGD, lr=1.0, batch_size=32, epochs=40
A3	+ AdamW optimizer	emb_size=650, hid_size=650, n_layers=2, dropout (embed+pre-proj), optimizer=AdamW, lr=0.001, batch_size=32, epochs=40
B1	Weight Tying only	emb_size=650, hid_size=650, n_layers=3, weight tying enabled, optimizer=SGD, lr=10, batch_size=64, epochs=100
B2	+ Variational Dropout	emb_size=650, hid_size=650, n_layers=3, weight tying, variational dropout, optimizer=SGD, lr=10, batch_size=64, epochs=100
B3	+ NT-ASGD (final)	emb_size=400, hid_size=1150, n_layers=3, weight tying, variational dropout, SGD→NT-ASGD, lr=30, batch_size=64, epochs=100
(Opt)	+ GloVe embeddings	emb_size=300, hid_size=1150, n_layers=3, GloVe (6B.300d), variational dropout, SGD→NT-ASGD, lr=30, batch_size=64, epochs=100

Additional Notes:

- Dropout placement in A2/A3 follows Zaremba et al., applied after embeddings and before projection layer.
- Variational Dropout (LockedDropout) in B2/B3 applied after embeddings and between LSTM layers.
- A-series models: gradient clipping=5, LR decay through `ReduceLROnPlateau` (factor=0.5, patience=2, min_lr=1e-4), eval batch size=128.
- B-series models: gradient clipping=0.25, weight decay=1.2e-6, patience=10, eval batch size=128.
- AdamW in A3 requires, thus uses significantly lower learning rate (0.001) compared to SGD variants (1.0-30.0).